

GemChat

Opis Struktury Projektu i Architektury

1. O Projekcie

GemChat to nowoczesna aplikacja mobilna na platformę Android, pełniąca rolę inteligentnego asystenta opartego na modelu językowym Google Gemini. Aplikacja oferuje możliwość prowadzenia konwersacji tekstowych oraz przesyłania obrazów (multimodalność) do analizy. Głównym założeniem aplikacji jest dbałość o prywatność (Privacy-by-Design) – wszelkie dane konwersacji są przechowywane lokalnie na urządzeniu użytkownika przy użyciu bazy danych Room.

2. Architektura Systemu

Projekt zrealizowano zgodnie z wzorcem architektonicznym **MVVM (Model-View-ViewModel)**, który jest aktualnym standardem zalecanym przez Google. Zapewnia to separację logiki biznesowej od warstwy prezentacji, ułatwia testowanie oraz skalowanie aplikacji.

- View (Warstwa Prezentacji):** Zbudowana w 100% z wykorzystaniem deklaratywnego interfejsu **Jetpack Compose**. Odpowiada za renderowanie interfejsu i reagowanie na interakcje użytkownika.
- ViewModel:** Pełni rolę pośrednika. Zarządza stanem aplikacji i komunikuje się z warstwą danych. Wykorzystuje *Kotlin Coroutines* do asynchronicznego pobierania danych bez blokowania głównego wątku (UI Thread).
- Repository (Warstwa Danych):** Orkiestrator zarządzający źródłami danych. Decyduje, czy pobrać/zapisać dane w lokalnej bazie (Room), czy uderzyć do zewnętrznego API (Gemini).
- Model:** Encje bazodanowe (Room SQLite) przechowujące lokalną historię wiadomości i metadane rozmów.

3. Stos Technologiczny

Technologia	Zastosowanie w projekcie
-------------	--------------------------

Kotlin	Główny, nowoczesny język programowania aplikacji
Jetpack Compose	Budowa deklaratywnego interfejsu użytkownika (zamiast XML)
Room Database	Lokalna baza danych (SQLite) do przechowywania historii czatów
Gemini REST API	Integracja z modelem AI (generowanie treści, analiza obrazu)
Coil	Asynchroniczne ładowanie i wyświetlanie obrazów
Navigation Compose	Zarządzanie przepływem i nawigacją pomiędzy ekranami

4. Struktura Katalogów Projektu

Kod źródłowy znajduje się w katalogu `app/src/main/java/com/gemchat/app/` i został logicznie podzielony ze względu na pełnione funkcje:

```
com.gemchat.app/  
├─ GemChatApplication.kt # Klasa główna aplikacji, inicjalizacja bazy i  
repozytorium  
├─ MainActivity.kt # Główny punkt wejścia, konfiguracja nawigacji  
├─ data/  
│   ├─ GemChatDatabase.kt # Konfiguracja bazy danych Room  
│   └─ dao/  
│       └─ ChatDao.kt # Interfejs (Data Access Object) dla zapytań SQL  
│   └─ model/  
│       └─ Entities.kt # Definicje tabel (Conversation, Message)  
│   └─ repository/  
│       └─ ChatRepository.kt # Pośrednik dostępu do bazy Room  
│       └─ GeminiRepository.kt # Obsługa zapytań sieciowych do API Gemini  
└─ ui/  
    ├─ about/ # Ekran z informacjami o aplikacji i autorach  
    ├─ chat/ # Ekran czatu (UI, logika wprowadzania, wyświetlanie dymków)  
    ├─ conversations/ # Główny ekran z listą wszystkich czatów  
    ├─ drawer/ # Komponenty bocznego menu nawigacyjnego  
    ├─ login/ # Ekran logowania (Mockup UI)  
    ├─ settings/ # Ekran ustawień (wybór modelu, zmiana motywu, czyszczenie  
danych)  
    ├─ splash/ # Ekran startowy z animacją wejściową  
    └─ theme/ # Konfiguracja kolorów, typografii i motywu aplikacji
```

5. Główne Ekrany i Funkcjonalności

- **Splash Screen:** Animowany ekran powitalny, pojawiający się podczas ładowania aplikacji.
- **Login Screen:** Interfejs logowania umożliwiający autoryzację (np. email, Google).
- **Conversations Screen:** Główny panel użytkownika wyświetlający historię zapisanych rozmów wraz z datą i podglądem ostatniej wiadomości. Zawiera również przycisk FAB do tworzenia nowej rozmowy.
- **Chat Screen:** Złożony widok konwersacji. Umożliwia pisanie zapytań, wysyłanie obrazków (integracja z galerią), wyświetlanie odpowiedzi AI oraz prezentację animacji "TypingIndicator" w oczekiwaniu na wynik z sieci.
- **Settings Screen:** Pozwala na zarządzanie preferencjami użytkownika (np. jasny/ciemny motyw, wybór modelu Gemini Flash/Pro, usunięcie historii).
- **About Screen:** Informacje o projekcie, autorach i wykorzystanych technologiach, z wbudowanym odtwarzaczem wideo (ExoPlayer).

6. Podsumowanie Oceny Rozwiązań

Projekt prezentuje zaawansowane użycie nowoczesnych standardów Androida. Przeniesienie całego interfejsu na **Jetpack Compose** oraz wdrożenie architektury **MVVM** z użyciem Kotlin Coroutines zapewnia wysoką responsywność i czystość kodu. Silnym atutem edukacyjnym jest nacisk na prywatność danych poprzez pełne lokalne cachowanie danych w bazie Room oraz bezpośrednią integrację z API modelu bez konieczności stawiania własnego serwera pośredniczącego.